# Slip 04

**Q1)**
**A**
**Explain Brute Force Attacks with an example, detailing Simple, Dictionary, and Hybrid brute force types.**

1. Definition of Brute Force Attack

A Brute Force Attack is a cybersecurity attack in which an attacker systematically tries different password combinations or possible keys until the correct one is found.

It is commonly used to guess weak passwords. The time required depends on password length, complexity, the number of possible combinations, and the system's security controls.

Example: If a password is a single digit from 0 to 9, an attacker may try 0, 1, 2, and so on until the correct digit is found.

2. Types of Brute Force Attacks

A. Simple Brute Force Attack

A simple brute force attack tries all possible combinations of characters until the correct password is found.

Working:

1. Select a possible password combination.
2. Compare it with the target password.
3. If it does not match, try another combination.

4. Repeat until a match is found or all possibilities are exhausted.

Example: For a four-digit numeric password, possible combinations range from 0000 to 9999, giving 10,000 possibilities.

Advantages:

- Can find the password if the search space is fully explored.
- Does not depend on knowing common password patterns.

Disadvantages:

- Can require considerable time and computing resources.
- Becomes impractical for long, complex passwords.

B. Dictionary Attack

A dictionary attack tries passwords from a predefined list of common passwords, words, and predictable combinations rather than testing every possible combination.

Working:

1. Prepare a list of likely passwords.
2. Select the first password from the list.
3. Compare it with the target.
4. Continue until a match is found or the list ends.

Example: An attacker might try common passwords such as password123, welcome123, and admin123.

Advantages:

- Faster than testing every possible combination when users choose common passwords.
- Requires fewer attempts when the correct password is in the list.

Disadvantages:

- Fails if the password is not included in the dictionary.
- Its success depends on the quality of the password list.

### C. Hybrid Brute Force Attack

A hybrid brute force attack combines dictionary-based guessing with additional characters, numbers, or symbols added to common passwords.

Working:

1. Select a common password from a dictionary.
2. Add likely numbers, symbols, or character variations.
3. Generate candidate passwords.
4. Test the candidates until a match is found or the search is exhausted.

Example: Starting with the common word `student`, possible candidates include:

- `student123`
- `student2026`
- `student@123`

Advantages:

- Can identify predictable variations of common passwords.

- Covers more possibilities than a basic dictionary attack.

Disadvantages:

- Requires more attempts than testing the original dictionary alone.
- May still fail against long, random passwords.

3. Difference Between the Three Types

| Basis | Simple Brute Force | Dictionary Attack | Hybrid Brute Force |
|---|---|---|---|
| Method | Tries all possible combinations | Tries passwords from a wordlist | Combines wordlist entries with variations |
| Password candidates | All combinations in the chosen character set | Common words and passwords | Common passwords plus numbers, symbols, or other changes |
| Speed | Often slow for complex passwords | Often faster for common passwords | Depends on the number of variations |

| | | | |
|---|---|---|---|
| Success depends on | Search space and available resources | Whether the password is in the list | Whether the password matches a generated variation |
| Example | Trying 0000 to 9999 | Trying password123 | Trying password123! |

4. Prevention of Brute Force Attacks

1. Use long, unique passwords or passphrases.
2. Avoid common words and predictable password patterns.
3. Enable multi-factor authentication (MFA).
4. Apply rate limiting and temporary account lockouts where appropriate.
5. Monitor repeated failed login attempts.
6. Store passwords securely using salted, computationally expensive password-hashing algorithms.

**Q2)**
**B**
**Write a Python program to demonstrate IP tracking simulation by storing fake IP addresses and printing location mapping using a dictionary**

```
# IP Tracking Simulation using Dictionary

ip_location = {
    "192.168.1.10": {"city": "Pune", "country": "India"},
    "192.168.1.20": {"city": "Mumbai", "country": "India"},
    "10.0.0.5": {"city": "Delhi", "country": "India"},
    "172.16.0.8": {"city": "Nagpur", "country": "India"},
```

```
    "192.168.2.15": {"city": "Chennai", "country": "India"}
}

print("Available IP Addresses:")
for ip in ip_location:
    print(ip)

target = input("\nEnter an IP address to track: ")

if target in ip_location:
    print("\nIP Address:", target)
    print("City:", ip_location[target]["city"])
    print("Country:", ip_location[target]["country"])
    print("Tracking Status: Location Found")
else:
    print("IP address not found in database!")
```